

Lec1

1-**Software engineering**: is defined as a process of analyzing user requirements and then designing, building, and testing software application which will satisfy those requirements

2-**Operational** :how good a software works on operations like budget , usability, efficiency, correctness ,functionality , dependability , security and safety.

3-**Transitional**: is important when an application is shifted from one platform to another

4-**DevOps** is a set of practices that combines software development (Dev) and IT operations (Ops).

5-“**Software maintenance** means to preserve from failure or decline.”

6-“**Software evolution** means a continuous change from lesser, simpler, or worse state to a higher or better state.”

7-Mark Halpern used the term **evolution** to define the dynamic growth of software a-evolution b-maintenance c-sw engineerinf d-none

8-**Continuing change (1st)** – A system will become progressively less satisfying to its user over time, unless it is continually adapted to meet new needs

9-**Increasing complexity (2nd)** – A system will become progressively more complex, unless work is done to explicitly reduce the complexity.

10-**Self-regulation (3rd)** – The process of software evolution is self regulating with respect to the distributions of the products and process artifacts that are produced.

11-**Conservation of organizational stability (4th)** – The average effective global activity rate on an evolving system does not change over time, that is the average amount of work that goes into each release is about the same.

12-**Conservation of familiarity (5th)** – The amount of new content in each successive release of a system tends to stay constant or decrease over time

13-**Continuing growth (6th)** – The amount of functionality in a system will increase over time, in order to please its users.

14-Declining quality (7th) – A system will be perceived as losing quality over time, unless its design is carefully maintained and adapted to new operational constraints.

15-Feedback system (8th) – Successfully evolving a software system requires recognition that the development process is a multi-loop, multi-agent, multi-level feedback system.

16-Some researchers and developers view preventive maintenance as a subset of perfective maintenance

17-Activities to make corrections: If there are discrepancies between the expected behavior of a system and the actual behavior

18-➤SCM is the discipline of managing and controlling change in the evolution of software system.

19-Software Re-engineering is a process of software development which is done to improve the maintainability of a software system.

20-Re-engineering is the examination and alteration of a system to reconstitute it in a new form.

21-**Re-engineering** is the reorganizing and **modifying existing** software systems to make them **more maintainable**

22-**Reverse engineering** is the activity of defining a more abstract, and easier to understand, representation of the system

23-“**forward engineering**,” is the traditional process of **moving from high-level abstraction and logical**

24-Advantages of Re-engineering:

- Reduced Risk**

- Reduced Cost**

- Better use of Existing Staff**

Lec2

25-**A legacy system** is an old system which is valuable for the company which often developed and owns it.

26-**Bennett used** the following definition: “large software systems that we don’t know how to cope with but that are vital to our organization.”

27- **Brodie and Stonebraker**: define a legacy system as “any information system that significantly resists modification and evolution to meet new and constantly changing business requirements.”

28- **Freeze**: The organization decides **no further work** on the **legacy** system should be performed.

29- **Outsource**: An organization may decide that supporting software is not core business and may outsource it to a specialist organization offering this service.

30- **Carry on maintenance**: Despite all the problems of support, the organization decides to carry on maintenance for another period

31- **Discard and redevelop**: **Throw all** the software away and **redevelop the application** once again from scratch

32- **Wrap**: It is a black-box modernization technique – surrounds the legacy system with a software layer that hides the unwanted complexity of the existing data.

33- **Migrate**: Legacy system migration basically moves an existing, operational system to a new platform.

34- **Impact analysis** is the task of **estimating the parts of the software** that can be affected if a proposed change request is made.

35- **Traceability analysis** : the high-level artifacts such as requirements, design, code and test cases related to the feature to be changed are identified.

36- **Dependency (or source-code) analysis**: attempt to assess the affects of change on semantic dependencies between program entities.

37- **Ripple effect analysis** measures the impact, or how likely it is that a change to a particular module may cause problems in the rest of the program.

38- **Measuring ripple effect** can provide knowledge about the system

(note)→39- The **change propagation activity** ensures that a change made in one component is propagated properly throughout the entire system.

40- **Refactoring** is the process of making a change to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior.

41- aims to **improve the internal structure** of the code, achieve through the **removal of duplicate code**, simplification, **making code easier** to understand called **Refactoring**

42- **Program understanding or comprehension** “ is the task of building mental models of an underlying software system.

43- **mental model** describes a programmer’s mental representation of the program to be comprehended

44- Program comprehension deals with the **cognitive processes** involved in constructing a mental model of the program.

45- **Hypotheses** are a way to understand code in an incremental manner.

46- **bottom up**: starting from the code.

47- **top down**: starting from high-level goal.

48- **Chunking** creates new, higher level abstraction structures from lower-level structures.

49- **Cross-referencing** means being able to make links elements of different abstraction levels.

(note)→50- **Chunking and cross-referencing** helps in building mental model of the program under study at different levels of abstractions.

51- Software reuse was introduced by **Dough McIlroy**: “The development of an industry **of reusable source-code software components** and the industrialization of the production

52- **Program families** are sets of programs whose common properties are **so extensive**

53- **domain analysis** is an activity of identifying objects and operations of a class of similar systems in a particular problem domain.

54- **Software reuse** is using **existing artifacts** or **software knowledge** during the **construction of a new software system**.

55- **data reuse**: involving a standardization of data formats.

56- **architectural reuse**: which consists of standardizing a set of design and programming conventions dealing with the logical organization of software.

57- **design reuse**: for some common business applications.

58- **program reuse**: which deals with reusing executable code.

59- **Software reuse is a way to increase**: software development productivity and quality of the software.

60- **Reusability** is a property of a software assets that indicates the degree to which it can be reused.

61- **Environmental independence**: The components can be reused irrespective of the environment from which they were originally captured.

62- **High cohesion** : The components that implement a **single operation** or set of related operations.

63- **Loose coupling** : The components that **have minimal links** to other components.

64- **Adaptability**: The components that are adaptable so they can be customized to fit a range of similar situation.

65- **Understandability** : The components which are easily understandable that users can quickly interpret functionality.

66- **Reliability**: The components that are **error-free**.

67- **Portability**: The components that are not restricted in terms of the software or hardware environment they operate in.

68- The **ISO/IEC 14764** standard defines **software maintenance**.

69- **Post-delivery activities** includes **changing software, providing training**, and operating a help desk.

70- **Pre-delivery activities** include planning for post-delivery operations.

71- A program producing a wrong output is an example of **processing failure**.

72- a program not being able to meet real-time requirements is an example of **performance failure**.

73- The process of corrective maintenance includes **isolation** and **correction** of defective elements in the software.

74- corrective maintenance is a **reactive** process.

75- **corrective maintenance** is performed **after** detecting defects with the system.

76- **Adaptive maintenance**: to enable the system to adapt to changes in its data environment or processing environment.

77- **Adaptive maintenance** includes **system changes, additions, deletions, modifications, extensions, and enhancements** to meet the evolving needs of the environment in which the system must operate.

78- **Changing the system** to support this **new hardware configuration** is one example **Adaptive maintenance**.

79- converting the system **from batch to on-line operation** is one example **Adaptive maintenance**.

80- changing the system to be compatible with other applications is one example **Adaptive maintenance**.

81- **Perfective maintenance**: is to make a variety of improvements, namely, user experience, processing efficiency, and maintainability.

82- the program outputs can be made more readable for better user experience. is one example **perfective maintenance**.

83- the program can be modified to make it faster, thereby increasing the processing efficiency. is one example **perfective maintenance**.

84- the program can be restructured to improve its readability, thereby increasing its maintainability. is one example **perfective maintenance**.

85- **perfective maintenance** include **restructuring of the code, creating and updating documentations**, and tuning the system to improve performance.

86- Which maintenance type is also called *reengineering*

A → **perfective maintenance**

87- **preventive maintenance** is to prevent problems from occurring by modifying software products.

88- **Preventive maintenance** is very often performed on safety critical and high available software systems.

89- **Training:** training the stakeholders about the implementation of the system

90- **Consultive** cost and length of time are estimated for maintenance work

91- **Evaluative:** include reviewing the program code and documentations, examining the ripple effect of a proposed change

92- **Reformative:** improve the readability of the documentation, make the documentation consistent with other changes in the system

93- **Update:** are substituting out-of-date documentation with up-to-date documentation, making semi-formal.

94- **Grooming:** are substituting components and algorithms with more efficient and simpler.

95- **Preventive:** perform changes to enhance maintainability and establish a base for making a future transition to an emerging technology.

96- **Performance:** produce results that impact the user.

97- **Adaptive:** port the software to a different execution platform, and increase the utilization of COTS components.

98- **Reductive:** drop some data generated for the customer, decreasing the amount of data input to the system, and decreasing the amount of data produced by the system

99- **Corrective:** are correcting identified bugs, adding defensive programming strategies, and modifying the ways exceptions are handled